Assignment 2

Ans. 1: I would rate myself according to the given A, B and C ranking as:

- LLM = C (I am working hard on understanding the various arenas of Artificial Intelligence and Machine Learning and came across computer vision side of it and want to learn more about LLMs and RAGs, applying my existing knowledge into this field.
- Deep Learning = B (Knows the crux about various techniques and how to use them for the particular use-case)
- AI = B
- ML = B

Ans. 2: LLMs stands for Large Language Models and the key architectural components for building a chatbot can be broken down into the following aspects as of my knowledge:

a. User Input: The user will give/ask about the queries and the input will be fed into the pipeline of the LLM.
b. Embeddings: The user input or documents are first divided into smaller chunks. These chunks are then converted into vector embeddings using an embedding model so that semantic similarity search can be performed efficiently.
c. Matching: The RAG architecture will then help to find out the most relevant answer for the user's question from the already built Vector Embeddings and will match from the most probable part of the contextual data provided as form of embeddings in the RAG based on the semantic search.
d. Ingesting into LLM: Then, only the most matched part of the answer from the RAG and the question will be fed into the LLM model which will then answer to the user according to the context rather than plain search from the Browser.
e. Output: Now the fresh and targeted output will be generated to the user in an augmented form.

Nowadays, most of the enterprise chatbots are using Retrieval-Augmented Generation (RAG), which enables the LLM to answer from the updated external knowledge instead of relying only on its pre-trained knowledge.

Ans. 3: Vector databases are basically the databases that are used to store Vector Embeddings which various tools or framework like LangChain or LlamaIndex has created from a huge amount of data provided to them. Think it is in this way, you have a huge amount of your company data i.e., Hiring procedure toolkit, employee's details, salary breakup, company's brochure, roles details, retirement policies, and many more various perks. Now, instead of providing the whole data at once, these frameworks will break that data into small chunks of data and will assign a numerical vector representation to each chunk of text which will consist of values mostly between -1 to 1 and arranged in such a way that each tokens having similar meaning or context will be so close in that embedding. The division of these chunks

can be entirely based on the developer either can be Fixed-size chunking, Sentence-based chunking, Semantic chunking etc. But this is going to affect the further RAG pipeline and output of the model.

Once the vector embeddings are ready, they need to be stored in a database and instead of just having a database for their mere storage, we can have a database that not only stores the embeddings but also kind-off have some semantic relation among the vectors. Such kind of database is known as Vector Database.

We require such databases in order to increase the similarity-based searches instead of exact matching phrases search. For eg: if someone search for “How to reset my password” but instead of these exact keywords the document contains “Steps to change your login credentials” then the basic keyword search mechanism will fail to do so but a similarity-based search can easily do the job.

Hypothetical problem and my selection of Vector Database:

Suppose I need to built a chatbot for a university that would answer the questions related to admissions, fee structure, courses, hostel facilities, faculties information, societies/programs, examination schedules and placement information.

Now instead of feeding this much amount of large data directly to a LLM model, I would store the vector embeddings in a vector database which would do all the similarity-based operations for me before sending the query to the LLM model.

For this, I would choose “Pinecone” because:

1. It can handle a large amount of data and can be integrated with python.
2. It makes the semantic search fast.
3. It scales well for large datasets.
4. It also supports metadata filtering making the retrieval even more efficient.
5. It can be integrated easily with various LLM frameworks like LangChain or LlamaIndex.

Although, other vector databases like FAISS, ChromaDB, Qdrant, Weaviate, Milvus, etc. are also widely used but I would choose Pinecone for a cloud-based chatbot since it is scalable, efficient, fast, simple and manageable.